Autonomous Production Choke Controller

Project Documentation

Honeywell Hackathon 2026

Engineering Submission

Document Reference:	**HON-APCC-2026-001**
Revision:	**2.0**
Date:	**2026-07-25**
Classification:	**Engineering Submission**

Table of Contents

Project Overview

The Autonomous Production Choke Controller is an industrial-grade predictive controller for oil well choke management. It implements a 5-phase control pipeline with FOPDT process modeling, brute-force candidate search, defense-in-depth safety, and automated system identification. All models are identified from simulator step tests only.

Architecture

5-phase pipeline: ProcessMonitor (perception, bias correction, SS detection), TargetManager (STARTUP->TRACKING->INFEASIBLE), Predictor (candidate generation, FOPDT Np=3, gain scheduling), SafetyGate (tightened constraints, emergency override), Selector (3-term cost, deadband, least-infeasible fallback).

Safety: 4-layer defense-in-depth. Layer 1: Proximity detection. Layer 2: Tightened constraints (+5% margin). Layer 3: Per-step trajectory checking. Layer 4: Emergency override (bypasses optimizer on hard limit violation).

Key Files

Table 1: Key source files.

File	Role
config.py	ControllerConfig + ConstraintLimits
models.py	9 data classes (FOPDT, WellModel, etc.)
controller.py	5-phase orchestrator
process_monitor.py	Perception + bias correction
target_manager.py	Mode state machine
predictor.py	Candidate generation + FOPDT prediction

safety_gate.py	Constraint enforcement + emergency override
selector.py	3-term cost optimization
simulator_adapter.py	TestSimulator + adapter
step_test.py	Automated step testing
model_identifier.py	63.2% FOPDT identification
plotter.py	Matplotlib visualization
run_scenarios.py	Main pipeline entry point
dashboard.py	Streamlit professional dashboard

Configuration

Table 2: ControllerConfig key parameters.

Parameter	Default	Description
ts	1.0 h	Control interval
ramp_limit	5.0%	Max choke move/step
prediction_horizon	3	Steps to predict
weight_effort	0.01	Effort penalty
weight_margin	0.005	Constraint margin reward
safety_margin	0.05	5% buffer
bias_alpha	0.3	EWMA bias factor
deadband	0.01	Micro-move suppression

How to Run

```
pip install -r requirements.txt
python run_scenarios.py
streamlit run dashboard.py
```

run_scenarios.py: Executes system identification -> model identification -> 3 control scenarios -> plot generation.
dashboard.py: Launches interactive monitoring application at localhost:8501.

Data Provenance

Table 3: Data provenance.

Source	Role
TestSimulator	System identification (6 step tests, 90 calls)
TestSimulator	Controller validation (3 scenarios, 153 calls)
Reference dataset	Post-hoc consistency check only (scratch/ directory)

All model parameters, controller tuning, and validation are derived exclusively from the Python simulator. The

reference dataset is never imported by any pipeline module and is used only for independent engineering consistency verification.